PATHS: Plug-and-Play Tracking from Heterogeneous Streams to a Shared Token Interface

Mengning Li
mli55@ncsu.edu
North Carolina State University
Raleigh, North Carolina, USA

Wenye Wang
wwang@ncsu.edu
North Carolina State University
Raleigh, North Carolina, USA

Abstract

Multimodal target tracking estimates a moving target's trajectory by fusing heterogeneous per-anchor feature streams from sensing modalities. Existing tracking pipelines are usually built for a fixed feature set and anchor layout. They become brittle when streams drop out, new feature families are introduced, or the deployment changes. We present PATHS, a deployment-aware token interface for plug-and-play tracking. PATHS converts per-anchor feature streams into a shared, fixed-width token interface, allowing a unified tracking model to operate as the available feature mix changes. To add a feature family, PATHS trains only a new encoder with a frozen reference decoder, aligning its tokens to the shared interface without paired cross-modal data. At runtime, set attention fuses any available tokens, and a shared decoder reconstructs the trajectory. A reference-frame mapping adapts PATHS to different spatial scales without retraining. Across seven feature families in simulation and three real deployments, PATHS achieves mean tracking errors of 0.13–0.49 m and reduces median error by up to 33% relative to fixed-input baselines under stream quality fluctuations.

CCS Concepts

• **Human-centered computing** → **Ubiquitous and mobile computing systems and tools**; • **Computer systems organization** → *Sensor networks.*

Keywords

Tracking, Multimodal Sensing, Sensor Fusion

ACM Reference Format:
Mengning Li and Wenye Wang. 2026. PATHS: Plug-and-Play Tracking from Heterogeneous Streams to a Shared Token Interface. In *The 32nd Annual International Conference on Mobile Computing and Networking (MobiCom '26), October 26–30, 2026, Austin, TX, USA.* ACM, New York, NY, USA, 16 pages. https://doi.org/10.1145/3795866.3844492

This work is licensed under a Creative Commons Attribution 4.0 International License.
MobiCom '26, Austin, TX, USA
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2505-0/2026/10
https://doi.org/10.1145/3795866.3844492

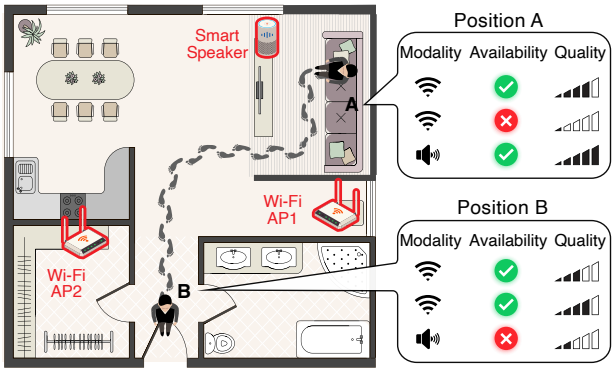


Figure 1: Feature availability and quality vary across space and time.

1 Introduction

1.1 Background and Motivation

Practical wireless sensing depends on a system's ability to adaptively leverage whichever sensing modalities are available at runtime. Despite recent progress, current solutions are often designed for specific modalities and require expert-led deployment, which restricts their scalability in unconstrained, everyday environments [1, 26, 28, 34, 38]. In real-world deployments, the presence and fidelity of wireless feature streams fluctuate significantly based on user movement and environmental changes, as illustrated in Figure 1 [18, 29]. Unlike traditional multimodal frameworks that assume a fixed and reliable sensor suite [22, 36], our objective is to enable robust tracking by effectively exploiting these pervasive but time-varying signal sources.

Current systems lack a unified tracking interface that remains usable as feature availability and deployment conditions change. In particular, most existing systems [10, 31, 32] follow a two-stage design. In the first stage, anchors extract location-related feature streams through modality-specific signal processing. In the second stage, these heterogeneous streams are fused over time to estimate trajectories. The first stage is naturally modality-specific. The second stage, however, is often not reusable. Classical approaches require feature-specific measurement models and careful tuning for

each anchor layout. Learning-based approaches are typically trained for a fixed input set and a fixed deployment.

This gap motivates a plug-and-play interface for tracking. Such an interface should accept any subset of available streams at runtime and avoid redesign for every feature combination. Building this interface is difficult for two reasons.

Diversity. A wide range of sensing modalities have been used for tracking, including Wi-Fi, millimeter wave (mmWave), and acoustic signals [38]. Each modality can expose multiple feature families. It is therefore impractical to build and maintain a separate pipeline for every possible combination.

Dynamic availability. Usable streams can change during tracking. A target may be observable through both acoustic and Wi-Fi signals at one location, but only through mmWave at another. The system must adapt to such changes without requiring a new design for each transition.

PATHS is designed to operate at the feature level rather than on raw wireless waveforms. It assumes that modality-specific front ends are available as upstream components. This scoping reflects how practical sensing systems are typically layered: physical-layer processing is handled by specialized per-modality modules, while PATHS provides a unified back end that fuses the resulting feature streams.

1.2 PATHS: Plug-and-Play Tracking through a Shared Token Interface

To address this problem, we present PATHS, a plug-and-play tracking framework built on a shared token interface. PATHS converts heterogeneous per-anchor feature streams into a shared token interface, allowing the same pipeline to operate as the available feature mix changes.

At a high level, PATHS attaches deployment context to each available stream, maps heterogeneous streams into a shared token interface, fuses any subset of the resulting tokens, and decodes the fused representation into a trajectory, providing a common tracking back end above modality-specific front ends (Sections 2 and 3).

Why this is challenging. A unified tracking interface must handle three coupled sources of variability. **First**, different feature families have different statistics and units, so the token semantics of different feature families must be aligned before fusion. **Second**, the set of available streams changes over time and may include missing, biased, or low-quality inputs, so fusion must remain robust under partial observability. **Third**, deployments differ in anchor placement and physical scale, so the same model should remain usable across sites without per-site redesign or retraining.

PATHS addresses these challenges through:

Decoder-anchored token alignment. Collecting training data for all modality and feature combinations is impractical. To add a new feature family, PATHS trains only its encoder against a reference decoder. This process encourages different encoders to emit compatible PATHS tokens.

Quality-aware, permutation-invariant fusion. At runtime, PATHS fuses a variable-size token set using masking and quality-aware gating. This design supports stable tracking when streams drop out or become unreliable.

Reference-frame mapping for deployment transfer. To generalize across deployments with different scales and layouts, PATHS maps each deployment into a reference coordinate frame through an affine transform, rescales features accordingly, runs the pretrained model in the reference frame, and maps the predicted trajectory back.

The individual ingredients of PATHS, modality-specific encoders, shared latent tokens, attention-based fusion with masking, and a sequence decoder, are established in multimodal learning [19, 25]; the novelty of PATHS lies at the system level rather than in a new fusion mechanism: onboarding new feature families without paired cross-modal data, deployment context without which a wireless feature sequence has no deployment-independent meaning (Figure 3), and reference-frame transfer across deployments.

1.3 Contributions

The key contributions of this paper are as follows:

- We present PATHS, a deployment-aware token interface that converts heterogeneous feature streams into a shared representation, enabling plug-and-play tracking under changing feature availability and anchor layouts.
- We introduce decoder-anchored token alignment, which adds a new feature family by training only its encoder against a reference decoder, avoiding paired cross-modal datasets while keeping the shared interface unchanged.
- We design a quality-aware, permutation-invariant fusion module and a reference-frame mapping that together support missing and low-quality streams as well as cross-deployment transfer, and we validate PATHS in both simulation and real-world deployments.

2 PATHS Overview

This section provides an overview of PATHS. We begin by formulating the problem and stating the design goals. We then introduce the main components of PATHS and summarize how they interact in the overall pipeline.

2.1 Problem Formulation

We study 2D trajectory tracking for a moving target observed through multiple anchors and feature families. PATHS operates on *extracted feature streams* rather than on raw wireless waveforms. At each time step, an anchor may provide zero or more streams from different feature families. Each stream includes a scalar feature value, an indicator of whether the

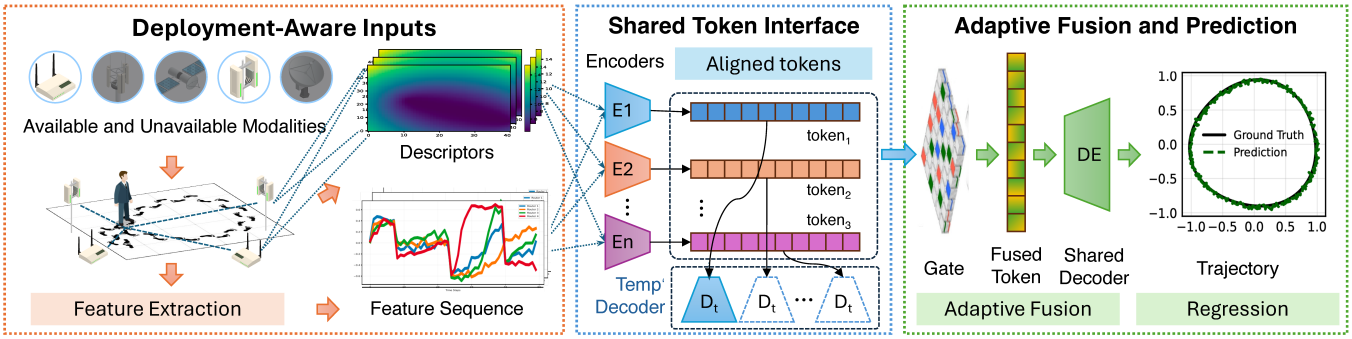


Figure 2: PATHS System Flow. Feature-specific encoders convert streams into a shared token interface. A fusion module and a decoder reconstruct the trajectory, while reference-frame mapping supports deployment transfer.

value is present, and a lightweight quality proxy. To support changing anchor layouts, each stream also carries a time-invariant deployment descriptor. This descriptor encodes information known *a priori*, such as feature identity and anchor geometry, and does not depend on the target location. The model therefore takes as input a set of per-stream feature observations together with their deployment descriptors.

PATHS converts these heterogeneous and intermittently available streams into a stable token interface. A feature-specific encoder maps each stream into a fixed-width token sequence. A permutation-invariant fusion module then aggregates the available tokens at each time step using masking and quality-aware gating. A shared decoder finally reconstructs the trajectory from the fused tokens. Because all streams are mapped to the same token interface, the pipeline can remain unchanged as the set of available streams varies.

We train the full pipeline end-to-end using supervised trajectory regression, with padded steps masked to support variable-length sequences. PATHS uses fixed-width tokens and variable-length sequences. It supports six representative feature families: angle of arrival (AoA), time of flight (ToF), bistatic path length (PL), path-length change rate (PLCR), time-of-flight rate (ToFR), and time-difference of flight (TDoF), plus a cross-anchor differential cue, differential of PLCR (dPLCR), while remaining open to additional location-related features. For ease of reference, Table 3 summarizes the main notation used throughout the paper.

2.2 Design Goals and Scope

PATHS is guided by four design goals: fidelity, interoperability, composability, and transferability.

2.2.1 Fidelity. PATHS should preserve the parts of each feature stream that actually matter for tracking. The interface should not reduce a useful signal to a coarse summary that is easy to store but no longer helpful for tracking. Absolute cues such as AoA and ToF should still retain enough information to support tracking in a global frame, while rate cues such as PLCR should preserve the temporal information needed to recover motion accurately over time.

2.2.2 Interoperability. PATHS should give different feature families a common interface. After encoding, streams with different raw forms and noise patterns should be consumable by the same downstream model. The goal is not to make the measurements identical, but to make their encoded outputs usable by one shared pipeline as the feature mix changes.

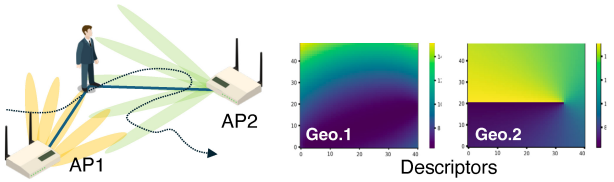
2.2.3 Composability. PATHS should work with any subset of available streams. The model should continue to operate when some inputs are missing, degraded, or newly added, without requiring a separate design for each feature combination. This reflects how real-world deployments behave: a stream may disappear, become noisy, or only be available over part of the trajectory, and the system should simply use the streams that remain informative.

2.2.4 Transferability. PATHS should remain usable across deployments with different anchor layouts and physical scales. The same tracking model should not have to be rebuilt for every new room or site. Although raw measurements are tied to physical frames, the underlying task is the same, so the back end should be reusable across deployments.

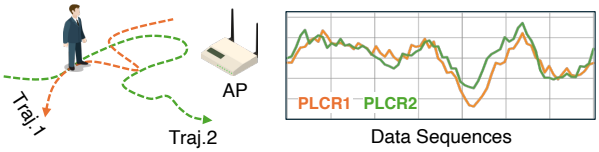
2.3 Functional Overview

Figure 2 provides an overview of PATHS, which has three main stages: deployment-aware input construction, tokenization into a shared interface, and adaptive fusion for trajectory prediction. We also use reference-frame mapping to support deployment transfer. We describe them in turn below.

2.3.1 Deployment-aware inputs. The left part of Figure 2 shows the input side of PATHS. Each sensing stream enters the system together with a deployment descriptor, rather than as a feature value alone. This descriptor specifies what the stream represents and where it is observed from, using installation-time metadata such as anchor coordinates, transmitter coordinates when applicable, and feature identity. As a



(a) The same trajectory can produce different measurements under different anchor layouts.



(b) Different trajectories can produce similar measurements under the same layout.

Figure 3: Data sequences alone are ambiguous; deployment descriptors resolve their physical meaning.

result, the same numeric stream can be interpreted correctly under different layouts. Importantly, PATHS starts *after* the front end that converts raw observations into feature streams, so its interface is defined at the feature level.

2.3.2 Tokenization into a shared interface. The middle part of Figure 2 shows how PATHS converts heterogeneous feature streams into a common token interface. Each feature family uses its own encoder, but all encoders emit token sequences in the same PATHS schema. Each per-step latent is a PATHS token. This allows heterogeneous inputs such as AoA, ToF, PL, and PLCR to be consumed by the same downstream tracking pipeline. During onboarding, a temporary reference decoder is used only in training to align a newly added encoder to this shared interface.

2.3.3 Adaptive fusion and trajectory prediction. The right part of Figure 2 shows the prediction path. At each time step, PATHS collects the available tokens, applies permutation-invariant fusion, and uses quality-aware gating to reduce the influence of degraded streams. The fused representation is then passed to a shared decoder, which outputs the target trajectory. The temporary reference decoder used during onboarding is not used at inference time.

2.3.4 Deployment transfer via reference-frame mapping. The mapping supports reuse across deployments with different layouts and scales. Before tracking, PATHS maps deployment-dependent coordinates and feature sequences into a reference frame. The same pretrained pipeline then runs in that frame, and the predicted trajectory is mapped back afterward. This keeps the tracking logic unchanged while reducing dependence on any single site geometry.

3 System Design

This section describes the design of PATHS. We present the feature-level training data simulator, deployment-aware encoders, decoder-anchored onboarding, adaptive fusion, the shared trajectory decoder, and reference-frame mapping.

3.1 Training Data Simulation

Figure 3 highlights a core ambiguity in localization: a raw feature sequence does not have a deployment-independent meaning. The same trajectory can induce different measurements under different anchor layouts, and similar-looking sequences can arise from different trajectories. Because PATHS operates on extracted features rather than on raw waveforms, we simulate at the *feature level*: the simulator generates trajectories, synthesizes clean feature streams analytically under known geometry, and then injects feature-space nonidealities. This matches the scope of PATHS and lets us study plug-and-play localization under controlled variations without claiming to fully model raw-signal propagation.

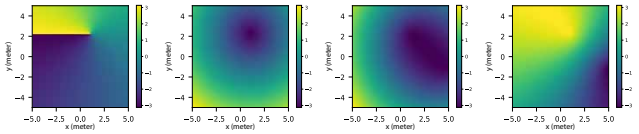
3.1.1 Trajectory generation. We simulate motion in a square workspace $[-L, L]^2$ with $L = 4$, using unit sampling interval $\Delta t = 1$. For each sample, the trajectory length is drawn from $T \sim \mathrm{Unif}\{50, \ldots, 200\}$. The target starts from a random location and follows a smooth bounded-velocity process. This yields diverse yet physically plausible trajectories.

3.1.2 Closed-form feature synthesis in simulation. Because PATHS begins after feature extraction, we synthesize clean feature streams directly from trajectory and deployment geometry. Figure 4 shows the spatial variation of different feature families, illustrating their complementarity for plug-and-play localization. For each sample, we place R=6 anchors at $\mathbf{p}_r = (x_r, y_r)$ drawn i.i.d. from $\mathrm{Unif}([-L, L]^2)$. For transmitter-dependent features, we draw $\mathbf{s} = (s_x, s_y)$ once per sample and keep it fixed for the full sequence.

Given a clean trajectory, we synthesize the corresponding feature streams in closed form. Specifically, AoA provides a bearing-like cue; ToF provides an absolute range-like cue; bistatic path length (PL) provides a bistatic distance cue; and TDoF provides a differential range-like cue. Rate-based features, including time-of-flight rate (ToFR) and path-length change rate (PLCR), are obtained by temporally differencing their corresponding absolute features. The cross-anchor differential feature dPLCR is further constructed by differencing PLCR across anchors at each time step.

3.1.3 Noise, bias, and other nonidealities. We simulate degradations that commonly arise in real-world deployments.

Zero-mean noise. We add independent zero-mean Gaussian noise to each feature family with feature-specific standard deviations (0.1745 rad for AoA, 0.15 m for ToF, and 0.05



(a) AoA. (b) ToF(R). (c) PL(CR). (d) TDoF.

Figure 4: Representative spatial patterns of synthesized feature values across feature families in simulation.

in native units for the remaining families). For angle-like signals, values are wrapped into $[-\pi, \pi]$ after perturbation.

Systematic bias: NLoS and multipath. To emulate non-zero-mean errors, we optionally inject an additive bias term $b_{t,i}$ on top of the clean value $y_{t,i}$, and then add zero-mean noise $\varepsilon_{t,i}$. The resulting observation is denoted by $y^{\text{noisy}}_{t,i}$. The bias may be stream-wise constant or slowly drifting to mimic time-varying multipath conditions.

Anchor coordinate uncertainty. Real-world deployments may contain anchor placement and survey errors. We therefore distinguish the *true* anchor coordinates $\mathbf{p}_r$, which are used to generate measurements, from the *assumed* coordinates $\tilde{\mathbf{p}}_r$, used when forming deployment metadata. We model $\tilde{\mathbf{p}}_r$ as $\mathbf{p}_r$ perturbed by a small i.i.d. Gaussian offset $\delta_r \sim \mathcal{N}(\mathbf{0}, \sigma^2_{\text{anch}}\mathbf{I})$.

Timing mismatch across streams. To study sensitivity to imperfect synchronization, we optionally apply a per-stream integer time shift $\Delta_i \in \mathbb{Z}$ to the measurement sequence *before* encoding. The encoder input at step t may therefore use the measurement originally generated at step $t+\Delta_i$. Out-of-range indices are treated as missing and masked out.

Outages and correlated failures. We synthesize availability masks $m_{t,i} \in \{0, 1\}$ using a two-state Markov process so that dropouts occur in short bursts. To emulate correlated degradation, such as occlusion affecting multiple modalities at once, we optionally introduce a global event state that simultaneously increases dropout probability and inflates noise or bias for a group of streams.

Quality proxy. When evaluating quality-aware gating, we also synthesize a proxy $q_{t,i} \in [0, 1]$ that correlates with observation quality, for example by mapping an SNR-like statistic derived from the injected noise level into $[0, 1]$.

3.1.4 Simulation scale and scope. Unless stated otherwise, each synthetic sample contains one trajectory of length $T \in \{50, \ldots, 200\}$, $R = 6$ anchors, and up to $6R = 36$ per-anchor candidate streams from the six base feature families {AoA, ToF, PL, PLCR, ToFR, TDoF}. We synthesize dPLCR separately as an optional cross-anchor cue because it is formed from anchor pairs rather than from a single anchor stream. The full synthetic dataset contains 80,000 training, 10,000 validation, and 10,000 test sequences.

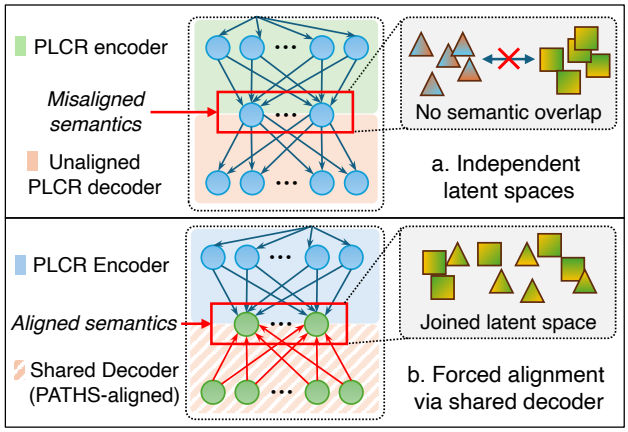


Figure 5: Decoder-anchored alignment for onboarding a new feature family.

The simulator therefore serves two roles in PATHS: it provides multi-feature supervision for alignment and fusion, and it supplies controlled stress tests for feature-space nonidealities such as bias, anchor uncertainty, and timing mismatch.

3.2 Feature-Aware Stream Encoders

PATHS converts heterogeneous streams into a shared token interface through deployment-aware encoding. Each stream is interpreted jointly with a time-invariant deployment descriptor and then mapped to a token sequence, since the same numeric feature values may carry different physical meanings under different layouts.

3.2.1 Deployment descriptor. For each stream, we define a time-invariant deployment descriptor by concatenating three pieces of metadata: (i) the system's assumed anchor coordinate, (ii) the transmitter coordinate when applicable, or a zero vector otherwise, and (iii) a learned feature-family embedding that identifies the feature type.

By default, we use $d_e = 8$, and the per-step encoder input dimension is $1 + 2 + 2 + d_e = 13$, consisting of one scalar feature value and its associated metadata. This descriptor is fully deployment-aware: it is determined by installation-time information such as device coordinates and feature identity, and does not depend on the unknown target.

3.2.2 Single-stream encoder architecture. For each stream, we form a per-step input vector $\mathbf{f}_{t,i}$ by concatenating the scalar feature value $y_{t,i}$ with its deployment descriptor $\mathbf{a}_i$. Each feature family uses the same encoder architecture: a two-layer bidirectional LSTM with hidden width 128 per direction and output width $C = 256$. The encoder output is then passed through a shared layer normalization module whose parameters are shared across feature families and anchors. This normalization helps keep numerical scales comparable across streams and simplifies downstream fusion.

3.2.3 Robustness augmentation. We apply light augmentation to both feature values and deployment metadata. This includes short-burst outages and quality jitter. When enabled, we also inject additive bias to emulate NLoS and multipath, perturb the assumed anchor coordinates used in the descriptor, and introduce small per-stream time shifts. These augmentations encourage the encoders to produce stable PATHS tokens under common deployment nonidealities.

At this stage, each encoder only maps its own input stream into a stable fixed-width representation conditioned on deployment context. A separate alignment step is still needed to make tokens from different feature families mutually compatible for downstream fusion and decoding.

3.3 Cross-Feature Token Alignment

To make tokens interoperable across feature families, PATHS aligns different encoders to a token interface through decoder reuse. The key idea is simple: once a reference decoder defines a decodable token interface, a newly added feature family only needs an encoder emitting tokens compatible with that decoder. Figure 5 illustrates the onboarding workflow: once a reference decoder defines a shared semantics frame, a new encoder can be trained to emit compatible tokens without modifying the back end.

3.3.1 Temporary reference decoder. We use a temporary decoder to define and probe a shared token interface during training. This decoder is used only for alignment and is discarded afterward. Its role is to test whether encoder outputs retain enough task-relevant information to support trajectory recovery. In this paper, we instantiate D_{tmp} as a two-layer LSTM followed by a linear head, although any supervised decoder that consumes encoder latents and predicts the downstream target can serve this role.

A natural question is why not simply reuse the runtime decoder. The runtime decoder consumes *fused* latents and is trained jointly with the fusion module (Sec. 3.4.3), yet the fusion module can only be trained on already-aligned tokens. D_{tmp} breaks this cycle because it consumes *single-family* token sequences, defining the interface before any fusion exists. AoA is the default reference family for its spatial coverage and cross-modality availability; residual bias toward it is absorbed when fusion and decoder later train end-to-end on multi-family tokens during joint optimization.

3.3.2 Two-stage alignment. Training proceeds in two stages:

(1) **Reference training.** We jointly train a chosen reference encoder, AoA by default, together with D_{tmp} using supervised trajectory regression. The loss is the average 2D localization error over time, with padded steps masked for variable-length sequences.

(2) **Encoder onboarding.** We then freeze the temporary decoder and train only the encoder for each additional feature family under the same supervision. Because D_{tmp} is fixed, the loss decreases only if the new encoder emits tokens that remain decodable in the same space.

If different feature encoders can each drive the same frozen D_{tmp} toward correct trajectory prediction, then their outputs are aligned at least to the extent required for downstream decoding. After alignment, D_{tmp} is discarded. This alignment is *functional* rather than semantic: tokens from different families are decodable for the same task by the same frozen probe, rather than semantically equivalent. What is aligned is the token *space*, not the decoder parameters, so discarding D_{tmp} does not undo it; the held-out-feature study in Sec. 4.2.4 tests this complete alignment chain directly.

3.3.3 Onboarding a new feature family. To add a new feature family, we instantiate only its encoder. We define the measurement and deployment descriptor as in Sec. 3.2.1, freeze D_{tmp}, and train the encoder with trajectory supervision so its tokens are decodable in the shared space. This supervision does not require samples paired with the reference family. The resulting tokens enter the existing interface without changing the downstream architecture.

This procedure lets PATHS incorporate a new feature family by training only its encoder in the lightweight onboarding setting, while keeping the rest of the localization pipeline architecturally unchanged. When maximum accuracy is required, the aligned interface can also be followed by optional joint fine-tuning of the final fusion and decoding stages. This rule extends beyond the seven families studied here: channels are classified by measurement semantics into angle-like, length-valued, and rate-valued types; phase, e.g., maps to the angle-like channel with standard wrapping and phase difference to the differential channel.

3.4 Adaptive Fusion with Quality Gating

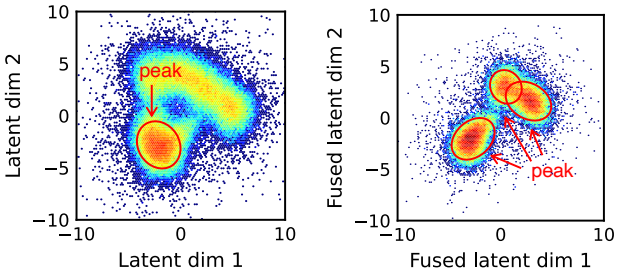
PATHS must operate with a changing set of input streams and remain stable when some of them are missing or degraded. We therefore use a permutation-invariant fusion module with explicit masking and quality-aware gating.

Figure 6 gives a qualitative view of the token interface before and after gating. Quality-aware fusion suppresses outlier streams and produces a more stable fused representation.

3.4.1 Fusion of variable-size token sets. At time t, we collect one token from each available stream and form the set

$$X_t = \{ \mathbf{z}_{t,i} \}_{i=1}^{n_t}, \quad \mathbf{z}_{t,i} \in \mathbb{R}^C. \tag{1}$$

We pad X_t to $n_{\max}$ tokens and maintain a binary availability mask $m_{t,i} \in \{0, 1\}$. When available, we also provide a quality proxy $q_{t,i} \in [0, 1]$. The fusion module maps this set to a



(a) Per-feature PATHS tokens. **(b) After gated fusion.**

Figure 6: Latent-space visualization of PATHS fusion.

single vector $\mathbf{z}_t \in \mathbb{R}^C$ and stacks $\{\mathbf{z}_t\}_{t=1}^T$ into $\mathbf{z}_{1:T} \in \mathbb{R}^{T\times C}$ for the shared decoder. Here $n_{\max}$ is a fixed padding budget equal to the maximum stream-pool size ($6R = 36$ by default). It does not vary with n_t, and slots beyond n_t are excluded from attention by the mask. Fusion is defined on sets rather than positional slots: each token's influence depends on its content, mask, and quality, so tokens from a new family enter as additional elements in the same aligned space.

3.4.2 Quality-aware gating. We first build cross-stream context using masked self-attention and then compute per-stream weights. Let $H_t \in \mathbb{R}^{n_{\max}\times C}$ denote the contextualized token matrix produced by attention, where masking ensures that missing tokens do not contribute. We compute normalized weights with a learnable temperature $\tau > 0$:

$$s_{t,i} = \mathrm{MLP}(H_{t,i}), \quad \alpha_{t,i} = \frac{m_{t,i}\, q_{t,i}\, \exp(s_{t,i}/\tau)}{\sum_j m_{t,j}\, q_{t,j}\, \exp(s_{t,j}/\tau)}. \quad (2)$$

Here τ is a learnable softmax temperature controlling the sharpness of the fusion weights [9]: small τ concentrates weight on the most reliable streams, large τ spreads it more uniformly. If quality is unavailable, we set $q_{t,i} \equiv 1$, so the learned score alone determines the weights. Quality metadata can refine gating but is not required for inference. Eq. (2) deliberately combines two complementary factors: the external proxy $q_{t,i}$ carries signal-level evidence from the front end, while the learned score $s_{t,i}$ is computed from *contextualized* tokens after cross-stream attention, so a stream that disagrees with the consensus of the others is down-weighted even when its own proxy is uninformative (Sec. 4.2.3). The fused token is then computed by weighted pooling followed by layer normalization:

$$\mathbf{z}_t = \mathrm{LN}\left(\sum_{i=1}^{n_{\max}} \alpha_{t,i}\, H_{t,i}\right). \quad (3)$$

3.4.3 Shared trajectory decoder. The shared decoder is the runtime module that interprets *fused multimodal* latents. It converts the fused latent sequence $\mathbf{z}_{1:T} \in \mathbb{R}^{T\times C}$ into the predicted trajectory $\hat{\mathbf{p}}_{1:T} \in \mathbb{R}^{T\times 2}$. We implement it as a two-layer LSTM with hidden size H, followed by a linear projection head. Because it is trained on fused latents rather than on single-family tokens, it can exploit cross-feature complementarity that the temporary decoder never sees.

Table 1: Model footprint and representative inference cost under the default setting. Latency is reported for batch size 1, T=100, and FP32 inference.

Component / setting	Value
Feature-id embeddings (d_e=8)	8 per feature
Feature encoders (2-layer BiLSTM)	541,696 per feature
Shared LayerNorm on tokens (C=256)	512
Fusion (self-attn + gating MLP + LN)	≈ 329,729
Shared decoder (2-layer LSTM + head)	329,986
Runtime total	4,452,155
Temp. decoder D_{tmp} (train-only)	≈ 0.46M
Latency (n_t=2/4/6)	43/75/107 ms per seq
Peak memory (n_t=6)	≈ 78 MB

We train the decoder with the same supervised trajectory regression objective used throughout the system. As summarized in Table 1, the decoder remains lightweight under the default setting. More importantly, its parameter count is independent of the number of anchors. Anchor-specific variability is handled by the shared encoders and the fusion module, while the decoder always receives the same fixed-width latent representation. This separation keeps the localization pipeline compact and makes it easier to scale to larger real-world deployments in practice.

3.5 Reference-Frame Mapping

To reuse a pretrained model across deployments with different origins, orientations, and scales, PATHS maps both deployment metadata and feature sequences into a canonical *reference space* (RS), runs the unchanged model in RS, and then maps the predicted trajectory back. The mapping changes coordinates and units while leaving the learned fusion and decoding functions untouched.

3.5.1 Affine canonicalization. Let $\mathbf{p} \in \mathbb{R}^2$ denote a 2D point in deployment coordinates. We map it to RS via

$$\mathbf{p}' = SR(\mathbf{p} - \mathbf{t}_0), \quad (4)$$

where $\mathbf{t}_0$ resets the origin, $R \in SO(2)$ aligns axes, and $S = \mathrm{diag}(s_x, s_y)$ normalizes spatial extent.

3.5.2 Deployment descriptors. Each stream is conditioned on a deployment descriptor containing the anchor coordinate, the transmitter coordinate when applicable, and a feature-id embedding. During transfer, we apply Eq. (4) to the coordinate components while keeping the feature embedding unchanged. This ensures that the encoder sees metadata in the same canonical frame used during training.

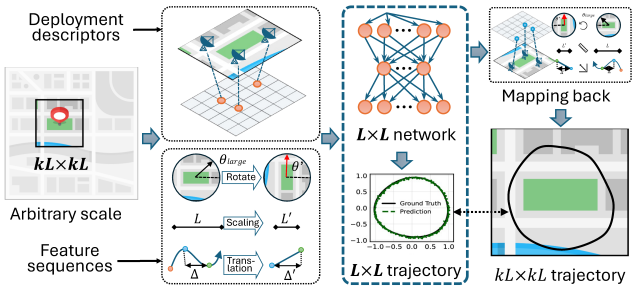


Figure 7: PATHS applies reference-frame mapping to align different scales and layouts for inference.

3.5.3 Feature sequences. Different feature families have different physical units, so they must be transformed according to their measurement semantics. Let s_ℓ denote the RS conversion factor for *length-valued* channels consistent with S; under uniform scaling, $s_\ell = s_x = s_y$. Let Δt_{train} and Δt_{test} denote the sampling intervals in training and deployment.

Angle (AoA). AoA should not be scaled. It only needs axis alignment with standard wrapping to $[-\pi, \pi]$:

$$\mathbf{u}(\theta) = [\cos\theta,\ \sin\theta]^\top, \quad \theta' = \operatorname{atan2}\big((R\,\mathbf{u}(\theta))_y,\ (R\,\mathbf{u}(\theta))_x\big). \tag{5}$$

Length-valued (ToF/PL/TDoF).

$$y' = s_\ell\, y, \qquad y \in \{\text{ToF, PL, TDoF}\}. \tag{6}$$

For relative quantities such as TDoF, we scale the value but do not introduce offsets.

Rate features (ToFR/PLCR/dPLCR).

$$\dot{y}' = s_\ell \frac{\Delta t_{\text{train}}}{\Delta t_{\text{test}}}\, \dot{y}, \qquad \dot{y} \in \{\text{ToFR, PLCR, dPLCR}\}. \tag{7}$$

An alternative is to resample sequences to the training rate, apply Eq. (6) to the corresponding absolute features, and then recompute them by differencing in RS.

3.5.4 Decoding and mapping back. The shared decoder outputs $\hat{\mathbf{p}}'_t$ in RS. We then invert Eq. (4) to return to deployment coordinates:

$$\hat{\mathbf{p}}_t = R^{-1} S^{-1} \hat{\mathbf{p}}'_t + \mathbf{t}_0. \tag{8}$$

In this way, deployment transfer reduces to semantics-aware unit normalization around an unchanged localization pipeline: map descriptors and feature sequences into RS, run the model once, and map the prediction back.

4 Evaluation

We evaluate PATHS in two complementary settings. We first use controlled simulation to isolate the effects of feature composition, stream availability, feature onboarding, and deployment mismatch. We then evaluate PATHS in real indoor deployments under both line-of-sight (LoS) and non-line-of-sight (NLoS) conditions across multiple sensing modalities.

4.1 Experimental Setup

4.1.1 Simulation Benchmarks. We generate synthetic benchmarks using the simulator described in Sec. 3.1. The workspace half-width is L=4, the number of anchors is R=6, and the trajectory length is drawn from $T \sim \text{Unif}\{50, \ldots, 200\}$. Each sample contains (i) noisy per-anchor, per-feature time series, (ii) a binary availability mask $m_{t,k,r}$, (iii) an optional quality proxy $q_{t,k,r}$, and (iv) the ground-truth trajectory.

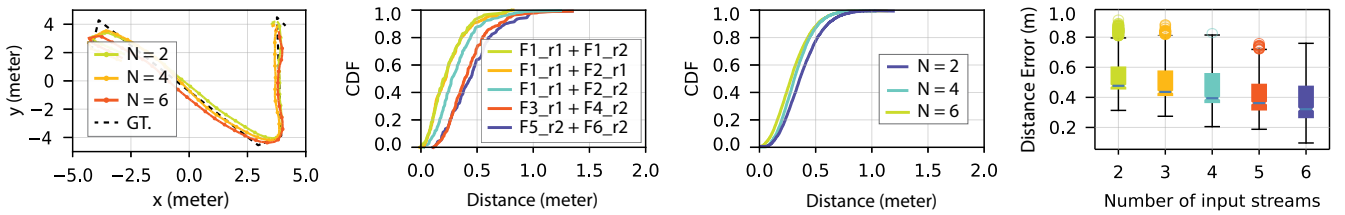
We synthesize the feature set as $\mathcal{K}$ ={AoA, ToF, PL, PLCR, ToFR, TDoF}. Rate features are generated by temporal differencing and then perturbed according to the default noise settings of Sec. 3.1.3. Outages are injected with a two-state Markov process. To evaluate robustness, we additionally create time-varying quality windows that inflate per-step noise, perturb assumed anchor coordinates, and optionally introduce per-stream timing mismatch. The synthetic dataset contains 80,000 training, 10,000 validation, and 10,000 test sequences generated with a fixed random seed. Random stream sampling draws from the default pool of $6R = 36$ streams.

4.1.2 Real-World Benchmarks. In addition to simulation, we collect an end-to-end real-world dataset spanning both LoS and NLoS indoor conditions across multiple layouts and scales. We evaluate PATHS on three sensing modalities: Wi-Fi, acoustics, and mmWave. Not every modality is available in every scene. Missing streams are handled directly by PATHS through masking and quality-aware fusion.

Modalities and feature extraction. For Wi-Fi, commodity access points (APs) and network interface cards (NICs) expose channel state information (CSI). We estimate AoA and ToF using standard subspace methods such as MUSIC [15, 21] and SAGE [7], and derive ToFR and PLCR from first-order differencing of range surrogates [11]. For acoustics, active chirp ranging provides ToF, and a calibrated microphone array provides AoA through beamforming. For mmWave, a synchronized joint radar–communication (JRC) station exposes AoA and ToF, while an unsynchronized receiver provides path-length-related features such as PL and PLCR.

For RF front ends, we first compensate carrier-frequency offset and device clock offsets using standard preamble-based estimation. We then align streams across modalities using timestamps, supplemented by hardware triggers when available, and resample them to a common step. Residual misalignment and slowly varying bias are not ignored; instead, they are stress-tested explicitly in simulation and handled by the robustness mechanisms of PATHS.

For quality-aware gating in real deployments, we use a lightweight per-stream proxy derived from signal-level statistics: for Wi-Fi, we use the subspace noise floor from the MUSIC estimation; for acoustics, we use the received SNR



(a) Trajectories vs. #streams. **(b) Error CDF by pairing.** **(c) Error CDF vs. #streams.** **(d) Error boxplots vs. #streams.**

Figure 8: Simulation results under varying numbers of input streams and feature compositions.

of the chirp response; for mmWave, we use the peak-to-sidelobe ratio of the range-angle spectrum. These proxies are normalized to $[0, 1]$ before being passed to the gating.

Deployment sites. Figure 11 shows the three deployment sites used in the real-world evaluation.

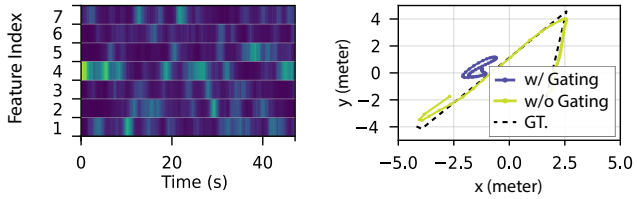
- **S1: Experimental Lab.** Area $\approx 100\,\mathrm{m}^2$. A mmWave JRC station and a receiver are deployed on opposite sides of the space. Reflectors are added to create rich multipath.
- **S2: Open Hall.** Area $\approx 400\,\mathrm{m}^2$. Several Wi-Fi routers and acoustic transmitter–receiver pairs are installed in the environment, with occasional structural pillars. The site includes mixed LoS and intermittent NLoS conditions.
- **S3: Multi-Room Office.** Area $\approx 50\,\mathrm{m}^2$. Three Wi-Fi routers are distributed across connected rooms. Data are collected both within and across rooms, with emphasis on NLoS conditions induced by walls and partitions.

Within each site, we instantiate multiple deployment settings spanning both single- and multi-modality configurations. Figure 11d illustrates a representative device layout.

4.1.3 Metrics. We report three trajectory metrics: (i) pointwise position error $\|\hat{\mathbf{p}}_t - \mathbf{p}_t\|_2$, (ii) average displacement error (ADE), which averages pointwise error over a trajectory, and (iii) final displacement error (FDE), which measures the error at the last step. CDFs are computed over pointwise errors pooled across trajectories. For the onboarding study, we additionally report the number of trainable parameters updated under each adaptation strategy.

4.1.4 Training Protocol. We train PATHS in three stages. First, we pretrain a reference encoder together with the temporary decoder for a selected base feature family. Second, we freeze the temporary decoder and align the remaining encoders to the same token interface, as described in Sec. 3.3. Third, we train the fusion module and the shared trajectory decoder end-to-end using the trajectory loss together with a lightweight regularizer for stable gating. All stages use synthetic data only. The real-world evaluation therefore measures transfer with fixed model parameters and no site-specific fine-tuning.

For the held-out-feature study, we first train a base model without the target feature family. We then compare three



(a) Time-varying weights. **(b) Trajectory recoveries.**

Figure 9: Effect of quality-aware fusion in simulation.

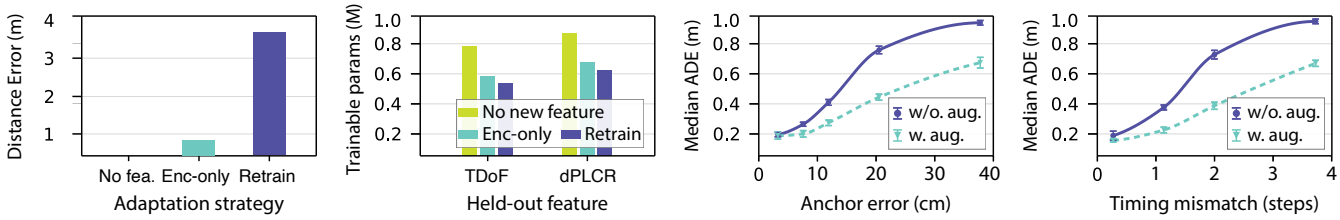
adaptation strategies: (i) *No new feature*, which leaves the base model unchanged, (ii) *Encoder-only onboarding*, which trains only the new feature encoder against the frozen temporary decoder, and (iii) *Full retraining*, which retrains the full model after adding the new feature family.

4.2 Simulation Results

We begin with controlled simulation to answer four questions: Can PATHS localize from small runtime-selected stream subsets? How much does it gain as more streams become available? Does quality-aware fusion improve robustness under degradation? And can PATHS support low-cost feature onboarding and moderate deployment mismatch? We address these questions in turn.

4.2.1 Two-Stream Localization. We first test whether PATHS can localize accurately from only two input streams chosen at runtime. For each simulated run, we sample two streams from the pool of per-anchor, per-feature inputs. The model is trained once and kept fixed throughout evaluation; no retraining is performed for different stream choices.

Figure 8c shows the CDF of pointwise error for input sizes $n \in \{2, 4, 6\}$. Even in the two-stream case, the 90th-percentile error remains below 0.8 m, indicating that PATHS can recover trajectories reliably from a small input set. Figure 8b further breaks down the two-stream setting by feature pairing. Pairs that combine complementary absolute cues, such as AoA+ToF, perform best. Mixed pairs that combine one absolute cue and one rate cue, such as AoA+PLCR, remain effective but degrade modestly because differencing amplifies noise and provides weaker constraints at the beginning of a trajectory. Rate-only pairs are the most difficult, since



(a) Held-out-feature error. **(b) Parameter budget.** **(c) Anchor positioning error.** **(d) Timing sensitivity.**

Figure 10: Feature-onboarding and mismatch-robustness studies in simulation.

single-step observations remain more spatially ambiguous and therefore rely more heavily on temporal consistency.

4.2.2 Multi-Stream Fusion. We next study how performance changes as more streams are available. For each run, we draw $n \in \{2, 4, 6\}$ streams uniformly at random from the full pool spanning all anchors and feature families. We then evaluate the same PATHS pipeline on 2,000 held-out trajectories.

Figure 8c shows a leftward shift in the error CDF as more streams are fused. The largest gain comes from moving from two to four streams. With only two streams, random sampling yields weak or redundant evidence. With four streams, the sampled set is more likely to contain complementary cues that resolve ambiguity early in the trajectory. Additional streams help beyond that point, but the marginal gain is smaller. Figures 8a and 8d show the same trend through representative trajectory overlays and error distributions.

4.2.3 Quality-Aware Fusion. We evaluate whether quality-aware fusion improves robustness when input streams become unreliable. At each time step and for each stream, we randomly assign one of three states: retained, corrupted with large noise, or dropped entirely. We then compare trajectory reconstruction with and without quality-aware gating while keeping the encoders and shared decoder fixed.

Figure 9a shows the resulting time-varying fusion weights. The gating module adapts to both instantaneous quality and short-term temporal context, suppressing streams that become unreliable. Figure 9b compares decoded trajectories under the same injected failures. Without gating, heavily corrupted inputs can dominate the fused representation and produce large trajectory deviations. With gating, the model places more weight on reliable streams and recovers a more stable trajectory despite outages and severe corruption. Its weights reflect both reported quality and each stream's agreement with the others. This remains useful when quality metadata is unavailable.

This behavior matters in practice. Many sensing systems continue to output measurements even after their quality has deteriorated substantially, and the downstream model is not always given an explicit reliability flag. Quality-aware fusion therefore provides a lightweight safeguard without requiring a different model for each failure pattern.

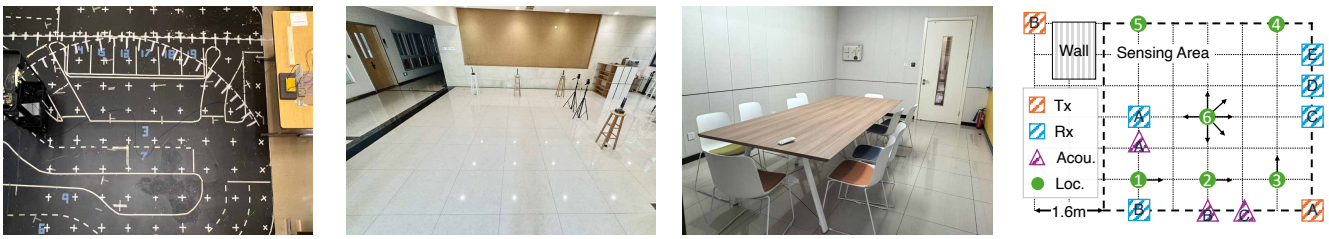
4.2.4 Encoder-Only Onboarding of New Feature Families. A central claim of PATHS is that a new feature family can be integrated by training only its encoder, while leaving the shared fusion and decoding stages unchanged. We validate this claim with a held-out-feature study. Specifically, we remove one feature family from base training, train the rest of the system as usual, and then compare three deployment-time options: *No new feature*, *Encoder-only onboarding*, and *Full retraining*. We choose TDoF and dPLCR as held-out families because they are structurally different from the reference family and represent two nontrivial additions: a transmitter-referenced differential cue and a cross-anchor rate cue.

Figure 10a shows the resulting localization error. Without onboarding, the base model cannot exploit the held-out cue. Encoder-only onboarding substantially reduces the error for both held-out families and recovers most of the gain obtained by full retraining. The remaining gap is small, indicating that decoder-anchored alignment produces tokens that remain compatible with the existing localization pipeline.

Figure 10b shows the corresponding trainable-parameter budget. Encoder-only onboarding updates only the new encoder, whereas full retraining updates the entire model. In our implementation, this reduces the number of trainable parameters by a factor of five while preserving most of the accuracy benefit. This result directly supports the modularity claim of PATHS: adding a new feature family is substantially cheaper than rebuilding the full pipeline.

4.2.5 Robustness to Geometry and Synchronization Mismatch. We next stress-test two practical deployment issues: mismatch between assumed and actual geometry, and imperfect synchronization across streams. We compare a model trained with robustness augmentation against one trained without it, while keeping the runtime architecture identical. At test time, we perturb the assumed anchor coordinates and, separately, apply per-stream integer timing offsets before encoding.

Figure 10c varies the anchor-coordinate error from small survey noise to much larger perturbations. Figure 10d varies the inter-stream timing mismatch from 0 to 4 steps. In both



(a) Experimental lab. (b) Open hall. (c) Meeting room. (d) Representative layout.

Figure 11: Real-world deployment sites used in the evaluation.

cases, localization error increases as mismatch becomes more severe, confirming that these deployment errors are first-order effects rather than benign nuisances. At the same time, the augmented model degrades more gracefully and maintains a clear advantage over the model trained without robustness augmentation across the tested range.

These curves show that PATHS remains usable under moderate anchor and synchronization error, and that simple training-time perturbations materially improve robustness. This makes the deployment story more credible than evaluating only the nominal, perfectly aligned case.

Taken together, the simulation results show that a single trained PATHS pipeline can (i) operate over changing feature subsets, (ii) benefit from additional streams when they become available, (iii) remain stable under missing or degraded inputs through quality-aware fusion, (iv) onboard a new feature family by training only its encoder, and (v) tolerate moderate deployment mismatch.

4.3 Real-World Results

We now turn to end-to-end real-world evaluation. The goal is to test whether the same interface remains effective when sensing conditions, feature availability, and deployment geometry vary across sites. We focus on representative feature compositions that range from strong absolute-cue combinations to more challenging rate-only settings.

4.3.1 Fusion in End-to-End Deployments. Figure 12 summarizes representative trajectory overlays and pooled error CDFs for four real-world feature compositions. All baselines [14, 23, 33] consume the same extracted feature streams, after the same front-end compensation of Sec. 4.1.2, and are evaluated on the same test sequences as PATHS; no baseline is tuned on the test sequences.

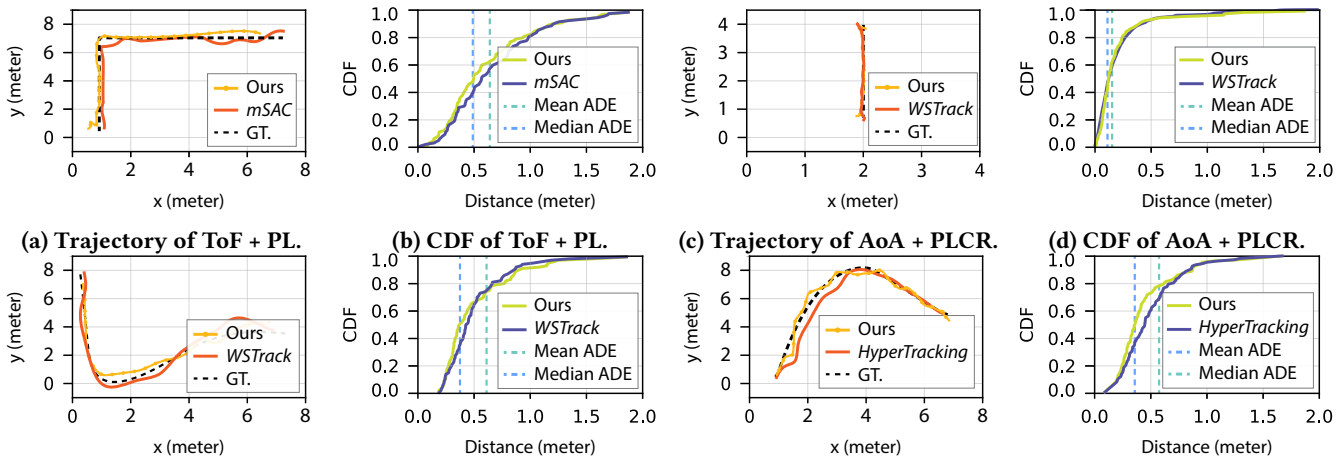
These compositions span distinct observability regimes. ToF + PL provides two absolute distance-like constraints, whereas AoA + PLCR combines an absolute direction with a temporal rate cue; PLCR and dPLCR alone rely mainly on motion continuity. Evaluating them through the same interface tests whether the token space retains the information needed by each regime without a feature-specific fusion back end. The trajectory overlays reveal temporal error accumulation, while the pooled CDFs summarize performance across sequences rather than a single representative path.

ToF + PL (mmWave). This setting combines two absolute distance-like cues. ToF provides an anchor-relative range-like measurement, while PL provides a bistatic path-length cue. Together they constrain position at each time step and help the model correct drift quickly when one cue becomes unreliable. We compare against mSAC [14]. Figures 12a and 12b show that PATHS follows the ground-truth trajectory closely and recovers quickly after brief deviations. Over 200 sequences, PATHS achieves a mean ADE of 0.62 m and a median ADE of 0.49 m, indicating stable fusion of complementary absolute geometric cues.

AoA + PLCR (acoustic + Wi-Fi). This setting combines an absolute angular cue with a rate cue. AoA stabilizes heading, while PLCR captures short-term path-length variation and helps distinguish turns and motion changes even when absolute range-like information is unavailable. We compare against WSTrack [23]. Figures 12c and 12d show that PATHS tracks the correct heading with low error at corners. Over 100 sequences, PATHS achieves a mean ADE of 0.15 m and a median ADE of 0.13 m. The trajectories remain stable through brief NLoS periods during which bearing quality degrades.

PLCR only (Wi-Fi). This is a deliberately difficult setting. PLCR is a rate feature, so it mainly reflects short-term motion rather than absolute position. Unlike settings with clean absolute cues such as AoA plus ToF, where closed-form geometric solvers are competitive, rate-only compositions admit no closed-form per-step position solution, because rate cues constrain only motion increments. As a result, localization relies more heavily on temporal consistency and on the motion prior encoded by the decoder. We compare against WSTrack [23]. Figures 12e and 12f show that PATHS still recovers the overall trajectory shape, and the errors are larger than in the mixed-cue settings. Over 100 sequences, PATHS achieves a mean ADE of 0.60 m and a median ADE of 0.39 m.

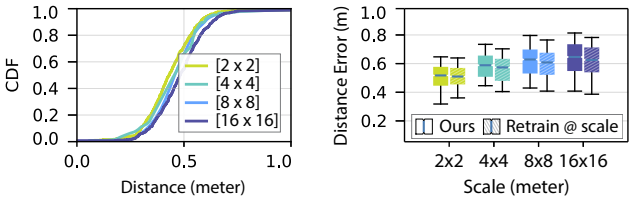
dPLCR only (Wi-Fi). This is the most difficult real-world setting we evaluate. dPLCR is a cross-anchor differential rate



(a) Trajectory of ToF + PL. (b) CDF of ToF + PL. (c) Trajectory of AoA + PLCR. (d) CDF of AoA + PLCR.

(e) Trajectory of PLCR only. (f) CDF of PLCR only. (g) Trajectory of dPLCR only. (h) CDF of dPLCR only.

Figure 12: Real-world trajectory recoveries and pooled error CDFs for representative feature compositions.



(a) CDFs of pointwise error. (b) Boxplots of ADE vs. scale.

Figure 13: Real-world scaling experiments.

cue that is especially sensitive to noise and provides weak absolute constraints at any single step. We compare against HyperTracking [33]. Figures 12g and 12h show that PATHS remains stable and follows the overall trend of the ground-truth trajectory, while the baseline deviates more visibly on difficult segments. Over 100 sequences, PATHS achieves a mean ADE of 0.53 m and a median ADE of 0.37 m.

The real-world results are consistent with the simulation trends. Feature compositions with stronger absolute cues perform best, while rate-only settings are more challenging but still feasible. Across all four cases, PATHS uses the same interface and the same core pipeline rather than requiring a separate fusion design for each feature combination.

4.3.2 Cross-Scale Transfer via Reference Mapping. We next test whether a model trained once at a base scale can be reused in rooms of different sizes without retraining. At deployment time, we apply an affine reference mapping to express the target site and its deployment descriptors in the base frame. The pretrained encoders, fusion module, and shared decoder then operate once in this normalized frame, and the prediction is mapped back by the inverse transform.

We evaluate square rooms of 2×2, 4×4, 8×8, and 16×16 m against retrain-at-scale models trained separately for each size. At all scales, the base model trained on 1×1 m data remains fixed; only reference-frame mapping is applied. Figure 13a shows that the error CDFs remain close across scales. The median ADE changes by at most ±3% across all trajectories, and the 90th–95th percentiles differ by at most 0.55 m. Figure 13b shows the same pattern through ADE boxplots. The model closely tracks the retrained baseline at each scale, with only mild widening of the error distribution as room size increases.

These results indicate that most scale variation is absorbed by geometric normalization rather than site-specific model adaptation at every new deployment.

5 Discussion

We next examine how PATHS extends beyond the evaluated setting and where its assumptions limit transfer.

5.1 Extending PATHS to Multiple Targets

This paper evaluates single-target tracking, yet much of PATHS is target-agnostic: the encoders, deployment descriptors, and reference-frame mapping depend only on installation-time metadata. What is missing is observation-to-target *association*; PATHS defines how streams are fused once assigned to a target, not how observations get assigned. Since fusion is permutation-invariant and the decoder is anchor-count independent (Table 1), a natural extension is an association module in front of PATHS maintaining a per-target token set; we scope multi-target tracking as future work.

5.2 Sim-to-Real Gap

All models are trained purely on synthetic data (Sec. 4.1.4), so the real-world results directly measure the simulation-to-reality gap: simulation ADE spans 0.13–0.49 m and the

real-world ADE 0.13–0.62 m across the four compositions of Sec. 4.3.1, with the same difficulty ordering in both regimes. This is also the most direct evidence against overfitting to synthetic noise: the 4.45M-parameter model (Table 1) carries its error profile over to measured data. The simulator is a feature-space stress-test generator, not a propagation model: real error signatures are feature-specific while our injected bias is per-stream additive (Sec. 3.1.3); feature-specific error structure is exercised by the deployments (S1–S3), and richer per-feature error models remain a limitation. No per-metric calibration is applied beyond the front-end compensation of Sec. 4.1.2; the quality proxies play that role at runtime.

5.3 Geometry Versus Propagation

The reference-frame mapping canonicalizes *geometry* and does not normalize *propagation*. As a coordinate transform it makes no assumption about room shape and is equally well defined for non-convex layouts; what non-convexity changes is multipath, which enters PATHS as feature-space bias and noise and is handled by robustness augmentation and quality gating (Sec. 3.4) rather than by the map. The intended use case is to train once in a canonical frame, in practice on simulated or previously collected data, and deploy to sites of different size, origin, or orientation with only a metadata transform. Cross-propagation-environment transfer, together with a deeper ablation of quality-proxy designs (Sec. 3.4), is left for future work. This keeps coordinate transfer explicit, while the learned model handles residual feature-level uncertainty.

6 Related Work

We review modality-specific tracking systems and cross-modal frameworks, focusing on the assumptions that distinguish them from PATHS in design.

6.1 Single-Modality Sensing and Tracking

Recent Wi-Fi work has broadened sensing beyond CSI-only pipelines: compressed beamforming reports enable sensing on standard devices [30], BFMSense demonstrates trajectory tracking from beamforming feedback matrices [35], and LocGPT studies how tailored pretraining improves transfer across indoor environments [39]. Tracking systems also push toward harder regimes, coupling tracking with gait recognition [24] or targeting device-free tracking in complex scenarios [17]. Most, however, remain tied to a particular RF observable or task family.

Acoustic systems reduce infrastructure assumptions while improving robustness: a single speaker can localize smart devices [4], ATP addresses multipath, Doppler shift, and sampling-frequency offset [3], AcouRadar localizes from a single sound source [5], MAVL combines multi-resolution AoA with room-structure inference under LoS and NLoS [27], and FusionTrack fuses CW and FMCW signals [37]. These pipelines remain coupled to chirp design, array or speaker geometry, and modality-specific compensation.

mmWave work increasingly exploits richer propagation structure: SuperSight achieves sub-centimeter NLoS localization for backscatter [2], Hydra models multi-bounce scattering for beyond-field-of-view radar sensing [16], SeRadar leverages secondary reflections under occlusion [8], and BiScatter integrates two-way radar backscatter communication and sensing [20]. Their downstream models remain tied to waveform-specific processing, aperture, and reflection geometry at individual deployment sites.

6.2 Cross-Modal and Unified Sensing

A smaller body of work combines modalities or seeks reusable abstractions: WSTrack fuses Wi-Fi and sound for device-free tracking [23], DuTrack corrects Wi-Fi drift with acoustics [12], Uni-Fi integrates multiple Wi-Fi sensing tasks in one architecture [13], and Babel studies modality alignment across Wi-Fi, mmWave, and depth [6]. These efforts either remain within one modality family or target broader human-sensing tasks rather than a deployment-aware token interface for localization.

Outside sensing, multimodal frameworks such as MulT [25] and attention bottlenecks [19] established the encoder–token–attention pattern that PATHS adopts. They assume a fixed modality set with paired cross-modal training data, carry no notion of deployment context (Figure 3), and have no counterpart to reference-frame transfer; PATHS departs from them on exactly these three axes rather than in the underlying attention-based fusion mechanism.

7 Conclusion

This paper studies plug-and-play tracking in realistic multimodal settings, where sensing streams may vary over time, new feature families may be introduced, and anchor layouts may differ across sites. To address this, we propose PATHS, a deployment-aware token interface. PATHS converts heterogeneous streams into shared fixed-width tokens by combining each stream with a deployment descriptor encoding feature identity and device geometry. It supports new feature onboarding without requiring paired cross-modal data. At inference time, PATHS fuses any available subset of tokens via permutation-invariant fusion with masking and quality-aware gating, and decodes the result into trajectories. It also uses reference-frame mapping to transfer across deployments with different layouts and scales. Experiments in simulation and real-world deployments show that PATHS remains effective under changing feature availability, benefits from complementary sensing streams, and transfers across affine-related layouts without retraining.

Acknowledgments

The NC State Plant Sciences Initiative supported this work.

References

[1] Iftikhar Ahmad, Arif Ullah, and Wooyeol Choi. 2024. WiFi-Based Human Sensing With Deep Learning: Recent Advances, Challenges, and Opportunities. *IEEE Open Journal of the Communications Society* 5 (2024), 3595–3623. doi:10.1109/OJCOMS.2024.3411529

[2] Kang Min Bae, Hankyeol Moon, and Song Min Kim. 2024. SuperSight: Sub-cm NLOS Localization for mmWave Backscatter. In *Proceedings of the 22nd Annual International Conference on Mobile Systems, Applications and Services (MOBISYS '24)*. Association for Computing Machinery, New York, NY, USA, 278–291. doi:10.1145/3643832.3661857

[3] Guanyu Cai and Jiliang Wang. 2024. ATP: Acoustic Tracking and Positioning under Multipath and Doppler Effect. In *IEEE INFOCOM 2024 - IEEE Conference on Computer Communications*. 1841–1850. doi:10.1109/INFOCOM52122.2024.10621165 ISSN: 2641-9874.

[4] Guanyu Cai and Jiliang Wang. 2024. Locating Your Smart Devices with a Single Speaker. In *Proceedings of the 22nd ACM Conference on Embedded Networked Sensor Systems*. ACM, Hangzhou China, 28–40. doi:10.1145/3666025.3699320

[5] Linsong Cheng, Zhao Wang, Yunting Zhang, Weiyi Wang, Weimin Xu, and Jiliang Wang. 2020. Acouradar: Towards single source based acoustic localization. In *IEEE INFOCOM 2020-IEEE conference on computer communications*. IEEE, 1848–1856.

[6] Shenghong Dai, Shiqi Jiang, Yifan Yang, Ting Cao, Mo Li, Suman Banerjee, and Lili Qiu. 2025. Babel: A Scalable Pre-trained Model for Multi-Modal Sensing via Expandable Modality Alignment. In *Proceedings of the 23rd ACM Conference on Embedded Networked Sensor Systems (SenSys '25)*. Association for Computing Machinery, New York, NY, USA, 240–253. doi:10.1145/3715014.3722068

[7] J.A. Fessler and A.O. Hero. 1994. Space-alternating generalized expectation-maximization algorithm. *IEEE Transactions on Signal Processing* 42, 10 (1994), 2664–2677. doi:10.1109/78.324732

[8] Danei Gong, Naiyu Zheng, Binbin Xie, Jie Xiong, Shuai Wang, Yuguang Fang, and Zhimeng Yin. 2025. SeRadar: Embracing Secondary Reflections for Human Sensing with mmWave Radar. In *Proceedings of the 31st Annual International Conference on Mobile Computing and Networking*. Association for Computing Machinery, New York, NY, USA, 231–246. https://dl.acm.org/doi/10.1145/3680207.3723471

[9] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the Knowledge in a Neural Network. *arXiv preprint arXiv:1503.02531* (2015).

[10] Manikanta Kotaru, Kiran Joshi, Dinesh Bharadia, and Sachin Katti. 2015. SpotFi: Decimeter Level Localization Using WiFi. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication*. ACM, London United Kingdom, 269–282. doi:10.1145/2785956.2787487

[11] Mengning Li and Wenye Wang. 2024. Hybrid Zone: Bridging Acoustic and Wi-Fi for Enhanced Gesture Recognition. In *IEEE INFOCOM 2024 - IEEE Conference on Computer Communications*. 981–990. doi:10.1109/INFOCOM52122.2024.10621132 ISSN: 2641-9874.

[12] Mengning Li and Wenye Wang. 2026. DuTrack: Long-Term Indoor Human Tracking with Dual-Channel Sensing and Inference. In *IEEE INFOCOM 2026 - IEEE Conference on Computer Communications*. IEEE. arXiv:2601.10972.

[13] Mengning Li and Wenye Wang. 2026. Uni-Fi: Integrated Multi-Task Wi-Fi Sensing. In *IEEE INFOCOM 2026 - IEEE Conference on Computer Communications*. IEEE. doi:10.48550/arXiv.2601.10980 arXiv:2601.10980.

[14] Mengning Li, Haocheng Zhu, Wenye Wang, and Eylem Ekici. 2025. mSAC: Enhancing Localization with mmWave Sensing and Orthogonal Signals. In *IEEE INFOCOM 2025 - IEEE Conference on Computer Communications*. IEEE, 1–10. doi:10.1109/INFOCOM55648.2025.11044619 ISSN: 2641-9874.

[15] Xiang Li, Shengjie Li, Daqing Zhang, Jie Xiong, Yasha Wang, and Hong Mei. 2016. Dynamic-MUSIC: accurate device-free indoor localization. In *Proceedings of the 2016 ACM International Joint Conference on Pervasive and Ubiquitous Computing*. ACM, Heidelberg Germany, 196–207. doi:10.1145/2971648.2971665

[16] Nishant Mehrotra, Divyanshu Pandey, Akarsh Prabhakara, Yawen Liu, Swarun Kumar, and Ashutosh Sabharwal. 2024. Hydra: Exploiting Multi-Bounce Scattering for Beyond-Field-of-View mmWave Radar. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking (ACM MobiCom '24)*. Association for Computing Machinery, New York, NY, USA, 1545–1559. doi:10.1145/3636534.3690710

[17] Xuanqi Meng, Weiping Ge, Yichen Tian, Xinyu Tong, Xiulong Liu, Xin Xie, and Wenyu Qu. 2025. MetaTrack: Enabling Wi-Fi Device Free Tracking in Complex Scenarios. *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.* 9, 4 (Dec. 2025), 200:1–200:26. doi:10.1145/3770686

[18] Majid Ghosian Moghaddam, Ali Asghar Nazari Shirehjini, and Shervin Shirmohammadi. 2023. A WiFi-Based Method for Recognizing Fine-Grained Multiple-Subject Human Activities. *IEEE Transactions on Instrumentation and Measurement* 72 (2023), 1–13. doi:10.1109/TIM.2023.3289547

[19] Arsha Nagrani, Shan Yang, Anurag Arnab, Aren Jansen, Cordelia Schmid, and Chen Sun. 2021. Attention Bottlenecks for Multimodal Fusion. In *Advances in Neural Information Processing Systems (NeurIPS)*. 14200–14213.

[20] Ryu Okubo, Luke Jacobs, Jinhua Wang, Steven Bowers, and Elahe Soltanaghai. 2024. Integrated Two-way Radar Backscatter Communication and Sensing with Low-power IoT Tags. In *Proceedings of the ACM SIGCOMM 2024 Conference (ACM SIGCOMM '24)*. Association for Computing Machinery, New York, NY, USA, 327–339. doi:10.1145/3651890.3672226

[21] Ralph Schmidt. 1986. Multiple emitter location and signal parameter estimation. *IEEE transactions on antennas and propagation* 34, 3 (1986), 276–280.

[22] Arindam Sengupta, Lei Cheng, and Siyang Cao. 2022. Robust Multiobject Tracking Using Mmwave Radar-Camera Sensor Fusion. *IEEE Sensors Letters* 6, 10 (Oct. 2022), 1–4. doi:10.1109/LSENS.2022.3213529

[23] Yichen Tian, Yunliang Wang, Ruikai Zheng, Xiulong Liu, Xinyu Tong, and Keqiu Li. 2023. WSTrack: A Wi-Fi and Sound Fusion System for Device-free Human Tracking. In *IEEE INFOCOM 2023 - IEEE Conference on Computer Communications*. IEEE, New York City, NY, USA, 1–10. doi:10.1109/INFOCOM53939.2023.10228898

[24] Xinyu Tong, Xiaoqiang Xu, Aiwen Yu, Xin Xie, Xiulong Liu, and Wenyu Qu. 2025. STAGR: Simultaneous Tracking and Gait Recognition With Commodity Wi-Fi. *IEEE Transactions on Mobile Computing* 24, 10 (Oct. 2025), 10868–10885. doi:10.1109/TMC.2025.3570993

[25] Yao-Hung Hubert Tsai, Shaojie Bai, Paul Pu Liang, J. Zico Kolter, Louis-Philippe Morency, and Ruslan Salakhutdinov. 2019. Multimodal Transformer for Unaligned Multimodal Language Sequences. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*. 6558–6569.

[26] Lei Wang, Tao Gu, Wei Li, Haipeng Dai, Yong Zhang, Dongxiao Yu, Chenren Xu, and Daqing Zhang. 2023. DF-Sense: Multi-user Acoustic Sensing for Heartbeat Monitoring with Dualforming. In *Proceedings of the 21st Annual International Conference on Mobile Systems, Applications and Services (MobiSys '23)*. Association for Computing Machinery, New York, NY, USA, 1–13. doi:10.1145/3581791.3596867

[27] Mei Wang, Wei Sun, and Lili Qiu. 2021. MAVL: Multiresolution analysis of voice localization. In *18th USENIX symposium on networked systems design and implementation (NSDI 21)*. USENIX Association, 845–858. https://www.usenix.org/conference/nsdi21/presentation/wang

[28] Tianben Wang, Zhangben Li, Xiantao Liu, Tao Gu, HongHao Yan, Jing Lv, Jin Hu, and Daqing Zhang. 2024. MultiResp: Robust Respiration Monitoring for Multiple Users Using Acoustic Signal. *IEEE Transactions on Mobile Computing* 23, 5 (May 2024), 3785–3801. doi:10.1109/TMC.2023.3279976

[29] Teng Wei and Xinyu Zhang. 2015. mTrack: High-Precision Passive Tracking Using Millimeter Wave Radios. In *Proceedings of the 21st Annual International Conference on Mobile Computing and Networking*. ACM, Paris France, 117–129. doi:10.1145/2789168.2790113

[30] Chenhao Wu, Xuan Huang, Jun Huang, and Guoliang Xing. 2023. Enabling Ubiquitous WiFi Sensing with Beamforming Reports. In *Proceedings of the ACM SIGCOMM 2023 Conference (ACM SIGCOMM '23)*. Association for Computing Machinery, New York, NY, USA, 20–32. doi:10.1145/3603269.3604817

[31] Jie Xiong and Kyle Jamieson. 2013. ArrayTrack: a fine-grained indoor location system. In *Proc. ACM MobiCom*.

[32] Jie Xiong, Karthikeyan Sundaresan, and Kyle Jamieson. 2015. ToneTrack: Leveraging Frequency-Agile Radios for Time-Based Indoor Wireless Localization. In *Proceedings of the 21st Annual International Conference on Mobile Computing and Networking*. ACM, Paris France, 537–549. doi:10.1145/2789168.2790125

[33] Xiaoqiang Xu, Xuanqi Meng, Xinyu Tong, Xiulong Liu, Xin Xie, and Wenyu Qu. 2023. HyperTracking: Exploring the Hyperbolic Model for Non-line-of-sight Device-free Wi-Fi Tracking. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies* 7, 4 (Dec. 2023), 1–26. doi:10.1145/3631434

[34] Dawei Yan, Panlong Yang, Fei Shang, Feiyu Han, Yubo Yan, and Xiang-Yang Li. 2025. Pushing the Limits of WiFi-Based Gait Recognition Towards Non-Gait Human Behaviors. *IEEE Transactions on Mobile Computing* 24, 7 (July 2025), 6137–6153. doi:10.1109/TMC.2025.3540863

[35] Enze Yi, Dan Wu, Jie Xiong, Fusang Zhang, Kai Niu, Wenwei Li, and Daqing Zhang. 2024. BFMSense: WiFi sensing using beamforming feedback matrix. In *21st USENIX symposium on networked systems design and implementation (NSDI 24)*. USENIX Association, Santa Clara, CA, 1697–1712. https://www.usenix.org/conference/nsdi24/presentation/yi

[36] Bojun Zhang, Mengning Li, Xin Xie, Luoyi Fu, Xinyu Tong, and Xiulong Liu. 2022. RC6D: An RFID and CV Fusion System for Real-time 6D Object Pose Estimation. In *IEEE INFOCOM 2022 - IEEE Conference on Computer Communications*. IEEE, London, United Kingdom, 690–699. doi:10.1109/INFOCOM48880.2022.9796802

[37] Jiarui Zhang and Jiliang Wang. 2024. FusionTrack: Towards accurate device-free acoustic motion tracking with signal fusion. *ACM Transactions on Sensor Networks* 20, 3 (2024), 71:1–71:30. doi:10.1145/3654666

[38] Jia Zhang, Rui Xi, Yuan He, Yimiao Sun, Xiuzhen Guo, Weiguo Wang, Xin Na, Yunhao Liu, Zhenguo Shi, and Tao Gu. 2023. A Survey of mmWave-Based Human Sensing: Technology, Platforms and Applications. *IEEE Communications Surveys & Tutorials* 25, 4 (2023), 2052–2087. doi:10.1109/COMST.2023.3298300

[39] Xiaopeng Zhao, Guosheng Wang, Zhenlin An, Qingrui Pan, and Lei Yang. 2024. Understanding Localization by a Tailored GPT. In *Proceedings of the 22nd Annual International Conference on Mobile Systems, Applications and Services (MOBISYS '24)*. Association for Computing Machinery, New York, NY, USA, 318–330. doi:10.1145/3643832.3661869

A Implementation Details

A.1 Training Setup

PATHS is trained in three stages. We first train a reference encoder together with a temporary decoder D_{tmp} using trajectory supervision. We then freeze D_{tmp} and train each new feature-family encoder to emit tokens in the same PATHS token interface. Finally, we discard D_{tmp} and train the fusion module plus the shared trajectory decoder on top of the aligned token interface.

For the held-out-feature study, we compare three options: (i) ignoring the new feature family, (ii) encoder-only onboarding with frozen D_{tmp}, and (iii) full retraining after adding the new feature family.

All stages use only simulator data from Sec. 3.1; no real-world measurements are used for training or fine-tuning.

A.2 Optimizer and Hyperparameters

All stages are trained with the Adam optimizer with learning rate 10^{-3}, batch size 64, weight decay 10^{-4}, and a cosine-annealing learning-rate schedule. Reference training uses 65 epochs, and each encoder-onboarding run uses 50 epochs. The loss in every stage is the masked average 2D localization error over time.

Architecturally, each feature encoder is a two-layer BiLSTM with hidden width 128 per direction and token width $C = 256$; the shared trajectory decoder is a two-layer LSTM with hidden size $H = 128$ and a linear head; D_{tmp} is a two-layer LSTM with a linear head. The feature-id embedding dimension is $d_e = 8$, the per-step encoder input dimension is 13, and the padding budget is $n_{\max} = 36$. Training-time robustness augmentation uses short-burst outages from the two-state Markov process of Sec. B.1, additive bias injection, anchor-coordinate perturbation, and per-stream time shifts.

A.3 Hardware Configuration

Training uses a single NVIDIA RTX 4090 GPU (24 GB) with PyTorch 2.7; Table 1 latency and memory are measured on a commodity laptop with batch size 1 and FP32 precision.

B Data and Preprocessing

B.1 Simulation Protocol

Unless stated otherwise, synthetic experiments use a square workspace $[-L, L]^2$ with $L = 4$, $R = 6$ anchors, and trajectory length $T \sim \text{Unif}\{50, \ldots, 200\}$. We synthesize $\mathcal{K} = \{\text{AoA}, \text{ToF}, \text{PL}, \text{PLCR}, \text{ToFR}, \text{TDoF}\}$, with rate features obtained by differencing their absolute counterparts. The simulator generates 80,000 training, 10,000 validation, and 10,000 test sequences. Outages are injected using a two-state Markov process with dropout probability 0.5 and persistence 0.8.

Table 2: Real-world sensing modalities. All streams are timestamp-aligned and resampled to a common step.

Modality	Front end	Features	Extraction
Wi-Fi	CSI from commodity APs / NICs	AoA, ToF, ToFR, PLCR, dPLCR	MUSIC (AoA); SAGE (ToF); differencing (rate cues)
Acoustics	Chirps + 6-microphone array	ToF, AoA	Matched-filter ranging; delay-and-sum beamforming
mmWave	JRC station + unsync. receiver	AoA, ToF, PL, PLCR	Range–angle spectrum; path-length estimates

B.2 Real-World Feature Extraction and Synchronization

Per-modality feature-extraction parameters. For Wi-Fi, CSI is collected from commodity APs and NICs. AoA is estimated with MUSIC, and ToF with SAGE; rate features (ToFR, PLCR, dPLCR) are obtained by first-order differencing of range surrogates at the common step. For acoustics, active ranging uses linear chirps of 18–22 kHz at a 48 kHz sampling rate; ToF is taken from the matched-filter peak, and AoA is obtained by delay-and-sum beamforming over a 6-microphone array. For mmWave, the JRC station operates at 24 GHz. AoA and ToF come from the range–angle spectrum, while the unsynchronized receiver provides PL and PLCR from path-length estimates over successive measurement steps.

Synchronization procedure. For RF front ends, we first compensate carrier-frequency offset and device clock offsets using standard preamble-based estimation. We then align streams across modalities using timestamps, supplemented by hardware triggers when available, and resample all streams to a common step. Steps for which a stream has no valid measurement after resampling are marked unavailable in the mask $m_{t,i}$ rather than interpolated. Residual misalignment is stress-tested explicitly in Sec. 4.2.5.

Quality proxies. The per-stream quality proxies are the subspace noise floor of the MUSIC estimate for Wi-Fi, the received SNR of the chirp response for acoustics, and the peak-to-sidelobe ratio of the range–angle spectrum for mmWave, each normalized to $[0, 1]$ by min–max normalization.

C Reference-Frame Mapping Example

Consider a deployment point $\mathbf{p} = (5, 3)^\top$ with translation $\mathbf{t}_0 = (1, 1)^\top$, rotation $R = \left[\begin{smallmatrix} 0 & -1 \\ 1 & 0 \end{smallmatrix}\right]$, and isotropic scaling $S = \mathrm{diag}(0.5, 0.5)$. Its mapped reference-space coordinate is

$$\mathbf{p}' = SR(\mathbf{p} - \mathbf{t}_0) = (-1, 2)^\top.$$

For an AoA observation $\theta = 30^\circ$, the rotated direction $R[\cos\theta, \sin\theta]^\top$ corresponds to $\theta' = 120^\circ$, so AoA is rotated but not scaled. For a length-valued feature $y = 6$ m and scale factor $s_\ell = 0.5$, $y' = s_\ell y = 3$ m. For a rate-valued feature $\dot{y} = 0.4$ m/step,

$$\dot{y}' = s_\ell \frac{\Delta t_{\text{train}}}{\Delta t_{\text{test}}} \dot{y}.$$

Predictions are mapped back to deployment coordinates by

$$\hat{\mathbf{p}} = R^{-1} S^{-1} \hat{\mathbf{p}}' + \mathbf{t}_0.$$

D Core Notation

Table 3 summarizes the notation used throughout.

Table 3: Core notation used in PATHS.

Symbol	Meaning
t, T	Time index; trajectory length.
i, n_t	Stream index; available streams at t.
$y_{t,i}$, $\mathbf{a}_i$	Measurement; deployment descriptor.
$\mathbf{f}_{t,i}$	Encoder input from $y_{t,i}$ and $\mathbf{a}_i$.
$\mathbf{z}_{t,i} \in \mathbb{R}^C$	Token from stream i at time t.
X_t	Token set at time t.
$m_{t,i}$, $q_{t,i}$, $\alpha_{t,i}$	Mask, quality, fusion weight.
$\mathbf{z}_t$, $\mathbf{z}_{1:T}$	Fused token; fused sequence.
$\hat{\mathbf{p}}_{1:T}$	Predicted 2D trajectory.
D_{tmp}	Temporary alignment decoder.